

Python Code

```
1  #Imports the library FreeSimpleGUI and assigns the alias of "GUI" for faster coding.
2  #E.g. "GUI." is equivalent to "FreeSimpleGUI."
3  import FreeSimpleGUI as GUI
4  GUI.theme('DarkBlue')
5
6  #Imports the FlightRadar24 API and SDK.
7  from fr24sdk.client import Client
8  #Takes the API token from a pre-defined environmental variable, via Windows
   PowerShell.
9  client = Client(api_token="0198c22c-876f-718e-bca1-
   a360872c8820|Y6pXXBQN250dYwViBPrqGsPWVr7q8AoPlyKczQBA832cb895")
10
11  #Other imports
12  import mysql.connector
13  import hashlib
14  from datetime import datetime, timedelta
15
16  #These two variables are used later on, in the "get_SearchFLT" function, as this API
   query
17  #requires a date-time range to search within and became inefficient having to change
   the date
18  #to meet the pre-requisites, this now does this for me and meets the 14-day max range.
19  TimeFrom = datetime.now()
20  TimeTo = TimeFrom + timedelta(days=14)
21
22  #Uses the MySQL connector in order to integrate the flightdeck database to my program.
23  db = mysql.connector.connect(
24      host = "localhost",
25      user = "root",
26      password = "",
27      database = "flightdeckdb"
28  )
29
30  #Quick reference messages for successful or unsuccessful database connection on start.
31  if db.is_connected():
32      print("DB - Connection Successful")
33  else:
34      print("DB - Connection Issue")
35
36  #Opens the cursor (a tool used to run MySQL queries and retrieve data).
37  Cursor = db.cursor()
38
39  #get_title1 getter procedure for title text.
40  def get_title1():
41      return [
42          [GUI.Text(text="FlightDeck", key="-TITLE-", expand_x=True, justification='c',
   font=("Courier New", 14))],
43          [GUI.HorizontalSeparator(color = "maroon")]
44      ]
45
```

```

46 #get_basic_create getter procedure for basic account creation layout.
47 #Includes all error message variants for the account creation page.
48 def get_basic_create():
49     return [
50         [GUI.Text(text="Create Account", expand_x=True, justification='c', font=
51 ("Courier New", 12))],
52         [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False, key='-
53 EMAIL_PER-')],
54         [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False,
55 password_char='*', key='-PASS_ENT_PER-')],
56         [GUI.Push(), GUI.Text('Re-enter Password'), GUI.Input(do_not_clear=False,
57 password_char='*', key='-PASS_REENT_PER-')],
58         [GUI.Text(text="Password must be at least 8 characters long.", expand_x=True,
59 justification='l', font=("Arial", 11), text_color="#de0000", key='-8CHAR_ERR_PER-',
60 visible=False)],
61         [GUI.Text(text="Passwords must match.", expand_x=True, justification='l',
62 font=("Arial", 11), text_color="#de0000", key='-PASS_MATCH_ERR_PER-',
63 visible=False)],
64         [GUI.Text(text="Account Creation Successful. Standby..", expand_x=True,
65 justification='l', font=("Arial", 11), text_color="#1ede00", key='-CREATED_MSG_PER-',
66 visible=False)],
67         [GUI.Text(text="This email address is already associated with an account.",
68 expand_x=True, justification='l', font=("Arial", 11), text_color="#de0000", key='-
69 EXIST_ERR_PER-', visible=False)]
70     ]
71
72 #get_commerce_sign getter procedure for basic sign in integrated into commercial sign
73 in layout.
74 def get_commerce_create():
75     return [
76         [GUI.Text(text="Create Account", expand_x=True, justification='c', font=
77 ("Courier New", 12))],
78         [GUI.Push(), GUI.Text('Company Domain (e.g. company.com)'),
79 GUI.Input(do_not_clear=False, key='-COMPANY-')],
80         [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False, key='-
81 EMAIL_CO-')],
82         [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False,
83 password_char='*', key='-PASS_ENT_CO-')],
84         [GUI.Push(), GUI.Text('Re-enter Password'), GUI.Input(do_not_clear=False,
85 password_char='*', key='-PASS_REENT_CO-')],
86         [GUI.Text(text="Password must be at least 8 characters long.", expand_x=True,
87 justification='l', font=("Arial", 11), text_color="#de0000", key='-8CHAR_ERR_CO-',
88 visible=False)],
89         [GUI.Text(text="Passwords must match.", expand_x=True, justification='l',
90 font=("Arial", 11), text_color="#de0000", key='-PASS_MATCH_ERR_CO-',
91 visible=False)],
92         [GUI.Text(text="Account Creation Successful. Standby..", expand_x=True,
93 justification='l', font=("Arial", 11), text_color="#1ede00", key='-CREATED_MSG_CO-',
94 visible=False)],

```

```

76         [GUI.Text(text="This email address is already associated with an account.",
expand_x=True, justification='l', font=("Arial", 11), text_color="#de0000"), key='-
EXIST_ERR_CO-', visible=False)]
77     ]
78
79     #Integrates the basic creation interface and the commercial creation interface into
one variable.
80     #Allows me to use the "enable_events" feature to track what account creation tab the
user is actively using
81     #Via the "-TAB_GROUP-" key identifier.
82     create_tabs = [
83         [GUI.TabGroup(
84             [[
85                 GUI.Tab("Personal", get_basic_create()),
86                 GUI.Tab("Commercial", get_commerce_create())
87             ]],
88             key="-TAB_GROUP-",
89             enable_events=True
90         )]
91     ]
92
93     #Create buttons with specific unique keys for the event-triggered dictionary inputs.
94     create_buttons = [
95         [GUI.Button('Create', bind_return_key=True)],
96         #The "Already have an account? Sign in." button has been assigned with the event
identifier "-GO_SIGN-" to skip validation
97         #and to close "window1" and create "window2", the same as if validation were to be
met when creating an account.
98         [GUI.Button('Already have an account? Sign in.', bind_return_key=True, key="-
GO_SIGN-")]
99     ]
100
101     #Combines all the sub-layouts into one main layout for the window1_run() function.
102     create_FLayout = get_title1() + create_tabs + create_buttons
103
104     #Password hashing algorithm.
105     def hash_password(password):
106         #Uses the hashlib SHA512 hash function to hash the entered password.
107         #uft-8 converts the password string into bytes as hashing functions cannot operate
on high-level text (only machine level).
108         return hashlib.sha256(password.encode('utf-8')).hexdigest()
109
110     def window1_run():
111         #Any functions / procedures regarding physical and device specific changes to the
window can only be made after
112         #it has been finalised using the procedure ".Finalize()"
113         window1 = GUI.Window('FlightDeck', create_FLayout, size = (700, 400),
enable_close_attempted_event = True).Finalize()
114
115         #Infinite loop that keeps the window event actively running.
116         while True:
117             #"create_values" is the dictionary created that contains inputs.
118             event, create_values = window1.read()
119             print(event, create_values)

```

```

120         active_tab = create_values['-TAB_GROUP-'] if "-TAB_GROUP-" in create_values
else None

121
122         #If an event is triggered by the user switching tabs, this condition ensures
false error messages aren't returned
123         #without an invalid input.
124         if event == '-TAB_GROUP-':
125             window1['-8CHAR_ERR_PER-'].update(visible=False)
126             window1['-PASS_MATCH_ERR_PER-'].update(visible=False)
127             window1['-8CHAR_ERR_CO-'].update(visible=False)
128             window1['-PASS_MATCH_ERR_CO-'].update(visible=False)
129             continue
130
131         #If the user attempts to close the window, a popup will appear (center screen)
to confirm their decision.
132         if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and
GUI.popup_yes_no('EXIT - Confirm you want to exit.') == 'Yes':
133             Cursor.close()
134             db.close()
135             break
136
137         #If the event identifier is equal to "-GO_SIGN-", which is set from the
"Already have an account? Sign in."
138         #button push on the interface, "window1" is closed and "window2" is created.
139         if event == "-GO_SIGN-":
140             window1.close()
141             window2_run()
142
143         #Assigns an index of the output dictionary "create_values" to different
equivalent variables using keys.
144         #One variable may have multiple keys, if the first key index is blank, it uses
the second.
145         password_per = create_values["-PASS_ENT_PER-"]
146         password_co = create_values["-PASS_ENT_CO-"]
147         password_reinput_per = create_values["-PASS_REENT_PER-"]
148         password_reinput_co = create_values["-PASS_REENT_CO-"]
149         email_address_per = create_values["-EMAIL_PER-"]
150         email_address_co = create_values["-EMAIL_CO-"]
151         company_domain = create_values["-COMPANY-"]
152
153         #Sets all validation variables / markers to "True", so that they can be
flipped if needed.
154         valid_8char_per = True
155         valid_8char_co = True
156         valid_passmatch_per = True
157         valid_passmatch_co = True
158         valid_domain = True
159
160         #Validates that the original password is longer than 8 characters.
161         if active_tab == 'Personal':
162             if len(password_per) < 8:
163                 window1['-8CHAR_ERR_PER-'].update(visible=True)
164                 valid_8char_per = False
165             if password_per != password_reinput_per:

```

```

166         window1['-PASS_MATCH_ERR_PER-'].update(visible=True)
167         valid_passmatch_per = False
168
169     #If the user is currently on the "Commercial" tab (create account window).
170     elif active_tab == 'Commercial':
171
172         #Selects the top record (only consisting of the CompanyDomain attribute)
from the
173         #"domains" table under the "flightdeckdb" database, where the
"CompanyDomain" is equal to
174         #the user-entered domain.
175         domain_check = "SELECT `CompanyDomain` FROM domains WHERE `CompanyDomain`
= %s"
176
177         Cursor.execute(domain_check, (company_domain,))
178         domain_result = Cursor.fetchone()
179
180         if len(password_co) < 8:
181             window1['-8CHAR_ERR_CO-'].update(visible=True)
182             valid_8char_co = False
183         if password_co != password_reinput_co:
184             window1['-PASS_MATCH_ERR_CO-'].update(visible=True)
185             valid_passmatch_co = False
186
187         #If a result is returned from the domain_check SQL query, the company
domain entered
188         #by the user must exist. Verification passed.
189         if domain_result:
190             event, create_values = window1.read(timeout=3000)
191             print("COMPANY DOMAIN CHECK SUCCESS")
192
193             valid_domain = True
194
195         else:
196             GlobalDomain = False
197
198         if (active_tab == "Personal" and valid_8char_per and valid_passmatch_per) or
(active_tab == "Commercial" and valid_8char_co and valid_passmatch_co and
valid_domain):
199
200             #If password validation is met (minimum 8 characters and both entries
matching)
201
202             #the password is hashed via the
203             #hash_password(password) function defined above.
204             hashed_password = hash_password(password_per or password_co)
205
206             # Inserts the email address, password and company name from the above
inputs into the column names
207             exists_check = "SELECT `EmailAddress` FROM credentials WHERE
`EmailAddress` = %s"
208
209             Cursor.execute(exists_check, (email_address_per or email_address_co,))
210             exists_result = Cursor.fetchone()
211
212             # stored in "values". The cursor then executes that operation with "sql1"
and "values" and is committed
213
214             #Checks if an account has already been created with the same email, if so,
the program will close.

```

```

209         if company_domain == "":
210             company_domain = None
211
212         if not exists_result:
213             sql1 = "INSERT INTO Credentials (`EmailAddress`, `Password`,
`CompanyName`) VALUES (%s, %s, %s)"
214             values = ((email_address_per or email_address_co), hashed_password,
company_domain)
215             Cursor.execute(sql1, values)
216             db.commit()
217             window1['-CREATED_MSG_PER-'].update(visible=True)
218             window1['-CREATED_MSG_CO-'].update(visible=True)
219             event, create_values = window1.read(timeout=3000)
220
221             window1.close()
222             window2_run()
223
224         else:
225             window1['-EXIST_ERR_PER-'].update(visible=True)
226             window1['-EXIST_ERR_CO-'].update(visible=True)
227             event, create_values = window1.read(timeout=2500)
228
229             window1.close()
230             break
231
232 #get_sign getter procedure for sign in layout.
233 #Includes all error message variants for the sign-in page.
234 def get_sign():
235     return [
236         [GUI.Text(text="Sign-In", expand_x=True, justification='c', font=("Courier
New", 12))],
237
238         [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False, key='-
EMAIL-')],
239         [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False,
password_char='*', key='-PASS-')],
240
241         [GUI.Text(text="Invalid Credentials.", expand_x=True, justification='l', font=
("Arial", 11), text_color="#de0000", key='-CREDS_ERR-', visible=False)],
242         [GUI.Text(text="Sign-In Successful. Standby..", expand_x=True,
justification='l', font=("Arial", 11), text_color="#1ede00", key='-SIGN_MSG-',
visible=False)]
243     ]
244
245 #Create buttons with specific unique keys for the event-triggered dictionary inputs.
246 sign_buttons = [
247     [GUI.Button('Sign In', bind_return_key=True)]
248 ]
249
250 #A singular tab has been used to group the get_sign() elements as I couldn't figure
out how to
251 #make them all group to the left-hand side of the window... this saved time...
252 sign_tab = [
253     [GUI.TabGroup(

```

```

254         [[GUI.Tab("Flight Information and Management System", get_sign())]]
255     )]
256 ]
257
258 sign_FLayout = get_title1() + sign_tab + sign_buttons
259
260 def window2_run():
261     #Declares the value of sign_values["-EMAIL-"] as a
262     #global variable for later use (in "window3_run").
263     global GlobalEmail
264
265     window2 = GUI.Window('FlightDeck', sign_FLayout, size = (700, 400),
266 enable_close_attempted_event = True).Finalize()
267     while True:
268         event, sign_values = window2.read()
269         print(event, sign_values)
270
271         if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and
272 GUI.popup_yes_no('EXIT - Confirm you want to exit.') == 'Yes':
273             Cursor.close()
274             db.close()
275             break
276
277     #Assigns an index of the output dictionary "sign_values" to different
278 equivalent variables using
279 #the equivalent keys.
280 password_check = sign_values["-PASS-"]
281 email_address_check = sign_values["-EMAIL-"]
282 #This executes a hash algorithm on the password entered on the sign-in page to
283 compare later on
284 #and since the original password has been hashed, it cannot be reverted to its
285 original state due to
286 #hashing being a one-way function.
287 hashed_password_check = hash_password(password_check)
288
289 #The "credentials_check" statement selects all records from the table
290 "credentials" where the email address
291 #and password are equal, respectively, to the "email_address_check" and
292 "hashed_password_check".
293 credentials_check = "SELECT * FROM `credentials` WHERE `EmailAddress` = %s AND
294 `Password` = %s"
295 Cursor.execute(credentials_check, (email_address_check,
296 hashed_password_check))
297 credentials_result = Cursor.fetchone()
298
299 #If no result is found, the credentials are deemed incorrect as they must be
300 correct initially at the
301 #create account stage. The credentials invalid error message is displayed by
302 settings its "visible"
303 #attribute to "True".
304 if not credentials_result:
305     window2['-CREDS_ERR-'].update(visible=True)
306
307 #If a result is found, the credentials are deemed correct. The sign-in
308 successful message ("-SIGN_MSG-") is

```

```

296         #displayed by setting its "visible" attribute to "True". Window3 is executed,
window2 is closed.
297     else:
298         window2['-SIGN_MSG-'].update(visible=True)
299         #3000ms window2 timeout to allow the user to read the message before
closing.
300         event, sign_values = window2.read(timeout=3000)
301         GlobalEmail = email_address_check
302
303         window2.close()
304         window3_run()
305
306
307 #Defines the "get_searchAPT()" function which searches for an airport.
308 def get_searchAPT(SearchEntApt):
309     #Uses "client" as the instance of the FlightRadar24 API key.
310     with Client(api_token="0198c22c-876f-718e-bca1-
a360872c8820|Y6pXXBQN250dYwViBPrqGsPWVr7q8AoPLYkCzQBA832cb895") as client:
311
312         #Declares the variable "airport" and uses the API key instance to return a
desired collection of
313         #attributes under the "get_light" pre-determined function, with the airport
ICAO code as a parameter.
314         airport = client.airports.get_light(code = SearchEntApt)
315
316         return airport
317
318 #Defines the "get_searchFLT()" function which searches for flight instances.
319 def get_searchFLT(SearchEntFlt):
320     with Client(api_token="0198c22c-876f-718e-bca1-
a360872c8820|Y6pXXBQN250dYwViBPrqGsPWVr7q8AoPLYkCzQBA832cb895") as client:
321
322         #Performs an API query using the "flight_positions.get_full()" function. This
query uses
323         #the user input, identifier "SearchEntFlt", as the value for the "flights"
parameter.
324         flight_positions = client.live.flight_positions.get_full(flights =
[SearchEntFlt])
325
326         #Performs an API query using the "flight_summary.get_full()" function. This
query uses the
327         #same user input as the value for the "flights" parameter.
328         flight_summary = client.flight_summary.get_full(
329             flights = [SearchEntFlt],
330             #The "TimeFrom" and "TimeTo" functions, part of the "datetime" library
331             #are used to automate the date/time range parameters.
332             flight_datetime_from = TimeFrom.isoformat(timespec='seconds'),
333             flight_datetime_to = TimeTo.isoformat(timespec='seconds'))
334
335         return flight_summary, flight_positions
336
337 #Groups the drop-down menu, under "Search for Information"
338 #into the respective search menus.
339 search_menus = [

```

```

340     [GUI.ButtonMenu('Drop-Down: Search for Flights / Airports',
341                     ['Menu', ['Flights::-FLT-', 'Airports::-APT-']],
342                     key="-MENU_OPT-")]
343 ]
344
345 #A list of elements that appear within the "Search for Information" tab. This allows
346 me to group them all
347 #under one collective list that can be used as a (GUI layout) parameter for the
348 "window3_run" function.
349 SearchElements = [
350     #Buttons
351     [GUI.Button('Search for Flight', key='-SEARCHBT_FLT-', bind_return_key=True,
352                 visible=False)],
353     [GUI.Button('Search for Airport', key='-SEARCHBT_APT-', bind_return_key=True,
354                 visible=False)],
355     [GUI.Button('Delete Alert (Double Click)', key='-ALERTDELETEBT-',
356                 bind_return_key=True, visible=False)],
357     #Tickboxes
358     [GUI.Checkbox('Show Stats For Nerds', default=False, key="-NERDCHK_FLT-",
359                 visible=False)],
360     [GUI.Checkbox('Create An Alert For This Flight', default=False, key="-
361     ALERTCHK_FLT-", visible=False)],
362
363     #Text and Inputs (each with a unique identifier).
364     [GUI.Push(), GUI.Text('Enter a flight number: ', key = "-SEARCHENT_FLT_TEXT-"),
365     GUI.Input(justification = "l", do_not_clear=False, key='-SEARCHENT_FLT-')],
366     [GUI.Push(), GUI.Text('Enter an airport IATA/ICAO code (e.g. LHR for Heathrow): ',
367     key = "-SEARCHENT_APT_TEXT-"), GUI.Input(justification = "l", do_not_clear=False,
368     key='-SEARCHENT_APT-')],
369     [GUI.Push(), GUI.Text('Enter the flight number of the alert you wish to delete: ',
370     key = "-ALERTDELETE_FLT_TEXT-"), GUI.Input(justification="l", do_not_clear=False,
371     key='-ALERTDELETE_FLT-')],
372
373 ]
374
375 #A list of tabs that appear within Window3, each taking the user
376 #to the respective place.
377 MainTabs = [
378     [GUI.TabGroup(
379         [[
380             #Tabs can include functions and other GUI elements
381             #within them (e.g. "get_title1()").
382             GUI.Tab("Manage Alerts", get_title1()),
383             GUI.Tab("Notifications", get_title1()),
384             GUI.Tab("Search for Information", search_menus)
385         ]],
386         #Allows the changing of tabs to act as an event trigger,
387         #with the identifier "-TAB_GROUP-"
388         key="-TAB_GROUP-",
389         enable_events=True
390     )]
391 ]
392
393 #These elements will display returns of API queries within respective tabs.

```

```

382 Base64 = ""
383 OutputElements= [
384     [GUI.Text(expand_x=True, justification='l', font=("Arial", 11), key="-AIRPORT-",
385         visible=False)],
386     [GUI.Text(expand_x=True, justification='l', font=("Arial", 11), key="-
387         FLIGHTSUMMARY-", visible=False)],
388     [GUI.Text(expand_x=True, justification='l', font=("Arial", 11), key="-
389         FLIGHTPOSITIONS-", visible=False)],
390     [GUI.Text(expand_x=True, justification='l', font=("Arial", 11), key="-NERD-",
391         visible=False)],
392     [GUI.Text(text="Alert Creation Successful", expand_x=True, justification='l',
393         font=("Arial", 11), text_color="#1ede00", key='-ALERTCREATED-', visible=False)],
394     [GUI.Text(text="This email address is already associated with an alert for this
395         flight.", expand_x=True, justification='l', font=("Arial", 11), text_color=
396         ("#de0000"), key='-ALERTEXISTS-', visible=False)],
397     [GUI.Text(text="", expand_x=True, justification='l', font=("Arial", 11), key='-
398         NOTIFICATION-', visible=False)],
399     [GUI.Image(Base64, key="-LOGO-", visible=False)]
400 ]
401
402 #Concatenates individual functions and element groups into one variable (see
403 "window3_run()").
404 main_FLayout = get_title1() + MainTabs + SearchElements + OutputElements
405
406 def window3_run():
407     window3 = GUI.Window('FlightDeck', main_FLayout, size = (700, 400),
408         enable_close_attempted_event = True).Finalize()
409     window3.maximize()
410
411     while True:
412         event, main_values = window3.read()
413         print(event, main_values)
414         active_tab = main_values['-TAB_GROUP-'] if "-TAB_GROUP-" in main_values else
415         None
416
417         #Assigns the "-SEARCHENT_FLT-" and "-SEARCHENT_APT-" identifier, within the
418         "main_values" dictionary,
419         #to the respective variables, as identifier cannot be passed into functions as
420         parameters.
421         SearchEntFlt = main_values['-SEARCHENT_FLT-']
422         SearchEntApt = main_values['-SEARCHENT_APT-']
423
424         if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and
425         GUI.popup_yes_no('EXIT - Confirm you want to exit.') == 'Yes':
426             Cursor.close()
427             db.close()
428             break
429
430         CompanyCheck = "SELECT `CompanyName` FROM credentials WHERE `EmailAddress` =
431         %s AND `CompanyName` IS NOT NULL"
432         Cursor.execute(CompanyCheck, (GlobalEmail,))
433         # Fetches one (the first) value.
434         CompanyCheckResult = Cursor.fetchone()

```

```

421
422     if CompanyCheckResult:
423         Logo = "SELECT `Logo` FROM domains WHERE `CompanyDomain` LIKE %s"
424         Cursor.execute(Logo, (CompanyCheckResult[0],))
425         #Fetches one (the first) value.
426         LogoResult = Cursor.fetchone()
427         Base64 = LogoResult[0]
428
429         window3["-LOGO-"].update(data=Base64, visible=True, subsample=3)
430
431     if active_tab == "Search for Information":
432
433         #Displays all GUI elements required to form an API query, to search for an
airport.
434         if main_values["-MENU_OPT-"] == "Airports::-APT-":
435             window3['-SEARCHENT_FLT_TEXT-'].update(visible=False)
436             window3['-SEARCHENT_FLT-'].update(visible=False)
437             window3['-SEARCHBT_FLT-'].update(visible=False)
438             window3["-NERDCHK_FLT-"].update(visible=False)
439             window3["-ALERTCHK_FLT-"].update(visible=False)
440
441             window3['-SEARCHENT_APT_TEXT-'].update(visible=True)
442             window3['-SEARCHENT_APT-'].update(visible=True)
443             window3['-SEARCHBT_APT-'].update(visible=True)
444             window3['-ALERTDELETE_FLT_TEXT-'].update(visible=False)
445             window3['-ALERTDELETE_FLT-'].update(visible=False)
446             window3['-ALERTDELETEBT-'].update(visible=False)
447
448
449             if event == '-SEARCHBT_APT-':
450                 #Triggers the "get_searchAPT" API query function with the user
input assigned to
451                 #"-SearchEntApt", assigned to a dictionary identifier, being passed
as a parameter.
452                 airport = get_searchAPT(SearchEntApt)
453                 #Updates the "-AIRPORT-" GUI element with the data stored in
"airport".
454                 window3["-AIRPORT-"].update(str("Airport Name:" + airport.name +
"\nAirport IATA:" + airport.iata), visible=True)
455
456                 #Displays all GUI elements required to form an API query, to search for a
flight.
457                 if main_values["-MENU_OPT-"] == "Flights::-FLT-":
458                     window3['-SEARCHENT_FLT_TEXT-'].update(visible=True)
459                     window3['-SEARCHENT_FLT-'].update(visible=True)
460                     window3['-SEARCHBT_FLT-'].update(visible=True)
461                     window3["-NERDCHK_FLT-"].update(visible=True)
462                     window3["-ALERTCHK_FLT-"].update(visible=True)
463
464                     window3['-SEARCHENT_APT_TEXT-'].update(visible=False)
465                     window3['-SEARCHENT_APT-'].update(visible=False)
466                     window3['-SEARCHBT_APT-'].update(visible=False)
467                     window3['-ALERTDELETE_FLT_TEXT-'].update(visible=False)
468                     window3['-ALERTDELETE_FLT-'].update(visible=False)

```

```

469         window3['-ALERTDELETEBT-'].update(visible=False)
470
471     if event == '-SEARCHBT_FLT-':
472         #Triggers the "get_searchFLT" API query function with the user
input
473         #assigned to "SearchEntFlt", being passed as a parameter.
474         flight_summary, flight_positions = get_searchFLT(SearchEntFlt)
475
476         # Ln 441-447: allows me to calculate a rough idea to whether a
flight is early / late.
477         ExpectedDuration = (flight_summary.data[0].circle_distance / 852)
+ 0.25
478         # By default, "Z" is included within the date/time string,
dictating UTC time.
479         # "Z" cannot be included for mathematical calculations, so is
replaced with a blank space.
480         ActualDuration = datetime.fromisoformat(
481             flight_positions.data[0].eta.replace("Z", "")) -
datetime.fromisoformat(
482             flight_summary.data[0].datetime_takeoff)
483
484         # the ".total_seconds()" / 3600 function and operation converts
485         # seconds into hours, for standardised units for calculation.
486         if (ActualDuration.total_seconds() / 3600) < ExpectedDuration:
487             status = "Likely Early"
488         if (ActualDuration.total_seconds() / 3600) > ExpectedDuration:
489             status = "Likely Late"
490
491         #Updates the respective GUI elements with the data from the API
query and sets their
492         #visibility to "True". Additional text has also been provided.
Labels, headings, etc.
493         #BROKEN: This doesn't appear, potential issue with visibility
after version restoration.
494         window3["-FLIGHTSUMMARY-"].update(str("-- Flight Info --" +
"\nAirline: " + flight_summary.data[0].operating_as + " // Flight: " +
flight_summary.data[0].flight + "\nAircraft: " + flight_summary.data[0].type + " //
Callsign: " + flight_summary.data[0].callsign + "\nDeparture Airport: " +
flight_summary.data[0].orig_iata + " // Departure Time: " +
flight_summary.data[0].datetime_takeoff), visible=True)
495         window3["-FLIGHTPOSITIONS-"].update(str("\nETA: " +
flight_positions.data[0].eta + "\n" + status), visible=True)
496
497         #If the "Show Stats For Nerds" checkbox is checked, then the
following additional
498         #attributes will be displayed, after a flight is searched for.
499         if main_values["-NERDCHK_FLT-"]:
500             window3["-NERD-"].update(str("\n-- Nerdy Stats --" +
"\nTracking and Data Source: " + flight_positions.data[0].source + "\nSquawk Code: " +
flight_positions.data[0].squawk + "\nAircraft Registration: " +
flight_positions.data[0].reg), visible=True)
501             if not main_values["-NERDCHK_FLT-"]:
502                 window3["-NERD-"].update(visible=False)
503

```

```

504                                     #If the "Create An Alert For This Flight" checkbox has been
selected...
505                                     if main_values["-ALERTCHK_FLT-"]:
506                                         ExistAlert1 = "SELECT `EmailAddress`, `FlightNumber` FROM
alerts WHERE `EmailAddress` = %s AND `FlightNumber` = %s"
507                                         #Executes the above SQL statement "ExistAlert1" and passes the
values of "email_address_per" /
508                                         #"email_address_co" and the flight number of the queried
flight into the respective parameters above.
509                                         Cursor.execute(ExistAlert1, (GlobalEmail,
flight_summary.data[0].flight))
510                                         #Fetches one (the first) value.
511                                         ExistAlertResult1 = Cursor.fetchone()
512
513                                     if not ExistAlertResult1:
514                                         AlertSql = "INSERT INTO alerts (`EmailAddress`,
`FlightNumber`, `DepartureTime`, `CreationTime`) VALUES (%s, %s, %s, %s)"
515                                         AlertValues = (GlobalEmail, flight_summary.data[0].flight,
flight_summary.data[0].datetime_takeoff, flight_positions.data[0].timestamp)
516                                         Cursor.execute(AlertSql, AlertValues)
517                                         #Commits the values to the database under one transaction.
518                                         db.commit()
519                                         Output = "Alert Creation Successful"
520                                         window3["-ALERTCREATED-"].update(Output, visible=True,
text_color=("#1ede00"))
521                                     else:
522                                         window3["-ALERTEXISTS-"].update(visible=True)
523
524                                     #Redundancy to ensure that respective GUI elements are hidden,
525                                     #should the user leave the tab.
526                                     else:
527                                         window3['-SEARCHENT_FLT_TEXT-'].update(visible=False)
528                                         window3['-SEARCHENT_FLT-'].update(visible=False)
529                                         window3['-SEARCHBT_FLT-'].update(visible=False)
530                                         window3["-NERDCHK_FLT-"].update(visible=False)
531                                         window3["-ALERTCHK_FLT-"].update(visible=False)
532
533                                         window3['-SEARCHENT_APT_TEXT-'].update(visible=False)
534                                         window3['-SEARCHENT_APT-'].update(visible=False)
535                                         window3['-SEARCHBT_APT-'].update(visible=False)
536                                         window3['-ALERTDELETE_FLT_TEXT-'].update(visible=False)
537                                         window3['-ALERTDELETE_FLT-'].update(visible=False)
538                                         window3['-ALERTDELETEBT-'].update(visible=False)
539
540                                     if active_tab == "Manage Alerts":
541                                         window3['-SEARCHENT_FLT_TEXT-'].update(visible=False)
542                                         window3['-SEARCHENT_FLT-'].update(visible=False)
543                                         window3['-SEARCHBT_FLT-'].update(visible=False)
544                                         window3["-NERDCHK_FLT-"].update(visible=False)
545                                         window3["-ALERTCHK_FLT-"].update(visible=False)
546
547                                         window3['-SEARCHENT_APT_TEXT-'].update(visible=False)
548                                         window3['-SEARCHENT_APT-'].update(visible=False)
549                                         window3['-SEARCHBT_APT-'].update(visible=False)

```

```

550         window3['-ALERTDELETE_FLT_TEXT-'].update(visible=True)
551         window3['-ALERTDELETE_FLT-'].update(visible=True)
552         window3['-ALERTDELETEBT-'].update(visible=True)
553
554         ExistAlert2 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
555         Cursor.execute(ExistAlert2, (GlobalEmail,))
556         # Fetches one (the first) value.
557         ExistAlertResult2 = Cursor.fetchone()
558
559         if ExistAlertResult2:
560             Separator = ""
561             # "len()" function determines the number of chars' within
562             # "ExistAlertResult2" to create a line break.
563             l = len(ExistAlertResult2[1]) * 2
564             for i in range(l):
565                 Separator = Separator + "-"
566
567             Output = ""
568             # Returns the 2nd, 3rd and 4th attribute within
569             # the returned "ExistAlertResult2" list.
570             for i in range(1, 5):
571                 # Concatenates the previous value of "Output" with the value
572                 # at index "i" from "ExistAlertResult2", plus a line break.
573                 Output = Output + str(ExistAlertResult2[i]) + "\n"
574             Output = Output + Separator
575             # Passes the "Output" as a parameter into the ".update()" function.
576             window3["-ALERTCREATED-"].update(Output, text_color="#FFFFFF",
visible=True)
577
578         else:
579             Output = "You have not created any flight alerts."
580             window3["-ALERTCREATED-"].update(Output, text_color="#FFFFFF",
visible=True)
581
582         if event == "-ALERTDELETEBT-":
583             SQLDelete = "DELETE FROM alerts WHERE `EmailAddress` = %s AND
`FlightNumber` = %s"
584             Cursor.execute(SQLDelete, (GlobalEmail, main_values["-ALERTDELETE_FLT-
"]))
585             db.commit()
586
587         else:
588             window3["-ALERTCREATED-"].update(visible=False)
589             window3["-FLIGHTSUMMARY-"].update(visible=False)
590             window3["-FLIGHTPOSITIONS-"].update(visible=False)
591
592         if active_tab == "Notifications":
593             ExistAlert3 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
594             Cursor.execute(ExistAlert3, (GlobalEmail,))
595             # Fetches one (the first) value.
596             ExistAlertResult3 = Cursor.fetchall()
597
598             SearchEntFlt = ExistAlertResult3[0][2]
599             flight_summary, flight_positions = get_searchFLT(SearchEntFlt)

```

```

600
601         if flight_summary.data[0].flight_ended:
602             window3["-NOTIFICATION-"].update("This flight has ended.",
visible=True)
603             return
604
605         #Ensures the two date-times are of the same format, for mathematical
operations.
606         TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))
607
608         #If current time is prior to the flight departure time (BEFORE
DEPARTURE)...
609         if datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z",
"")) < ExistAlertResult3[0][3]:
610
611             #If the time until departure is less than 4 hours...
612             if TDeparture < timedelta(hours=4):
613
614                 #If no alert has been previously "pushed" for this flight...
615                 if ExistAlertResult3[0][5] is None:
616                     #Flight departure time - current time = time until departure
(via "timestamp"
617                     #attribute, under the FlightRadar24 API "flight_positions"
endpoint).
618                     window3["-NOTIFICATION-"].update("Time Until Departure: " +
TDeparture, visible=True)
619                     TAlert = None
620
621                     #Updates the "LastAlerted" attribute with the current
timestamp.
622                     UpdateAlertSql = "UPDATE alerts SET `LastAlerted` = %s WHERE
`EmailAddress` = %s"
623                     UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
624                     Cursor.execute(UpdateAlertSql, UpdateAlertValues)
625                     db.commit()
626
627                 #Else (if an alert has been previously "pushed" for this
flight)...
628                 else:
629                     #TAlert (time since last alert) is calculated.
630                     TAlert =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][5]))
631
632                     #If the time since the last alert (TAlert) is >= 20 minutes...
633                     if TAlert >= timedelta(minutes=20):
634
635                         #An up-to-date API query is made.
636                         flight_summary, flight_positions =
get_searchFLT(SearchEntFlt)

```

```

637             #Calculates an up-to-date "TDeparture" and ensures the two
date-times
638             #are of the same format, for mathematical operations.
639             TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))
640
641             #Outputs "TDeparture"
642             window3["-NOTIFICATION-"].update("Time Until Departure: "
+ TDeparture, visible=True)
643
644             #Updates the "LastAlerted" attribute with the current
timestamp.
645             UpdateAlertSql = "UPDATE alerts SET `LastAlerted` = %s
WHERE `EmailAddress` = %s"
646             UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
647             Cursor.execute(UpdateAlertSql, UpdateAlertValues)
648             db.commit()
649
650             #If the time until departure is greater than 4 hours...
651             if TDeparture > timedelta(hours=4):
652                 #Outputs statement.
653                 window3["-NOTIFICATION-"].update("Alerts only appear 4 hours in
advance to departure" + "(" + TDeparture + ")", visible=True)
654
655             #Else (if current time is beyond the flight departure time (AFTER
DEPARTURE))...
656             else:
657                 if ExistAlertResult3[0][5] is None:
658                     TAlert = None
659                 else:
660                     TAlert =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][5]))
661
662                 if TAlert is not None and TAlert >= timedelta(minutes=20):
663
664                     #An up-to-date API query is made.
665                     flight_summary, flight_positions = get_searchFLT(SearchEntFlt)
666                     #Calculates an up-to-date "TDeparture" and ensures the two date-
times
667                     #are of the same format, for mathematical operations.
668                     TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))
669
670                     #Converts elapsed flight time into hours.
671                     ElapsedTime = (flight_summary.data[0].flight_time / 60 / 60)

```

```

672         InFlightInfo = "-- Journey Data --" + "\nDeparture Time: " +
flight_summary.data[0].datetime_takeoff + "\nETA: " + flight_positions.data[0].eta +
"\nElapsed Time: " + str(ElapsedTime) + " hrs" + "\nElapsed Distance: " +
str(flight_summary.data[0].actual_distance) + "\n\n-- Position / Navigation Data --" +
"\nAltitude: " + str(flight_positions.data[0].alt) + " ft" + "\nHeading: " +
str(flight_positions.data[0].track) + "\nIndicated Airspeed: " +
str(flight_positions.data[0].vspeed) + " kts" + "\nLatitude/Longitude: " +
str(flight_positions.data[0].lat) + " / " + str(flight_positions.data[0].lon) +
"\n\nTo Date As Of " + flight_positions.data[0].timestamp
673         window3["-NOTIFICATION-"].update(InFlightInfo, visible=True)
674
675         #Updates the "LastAlerted" attribute with the current timestamp.
676         UpdateAlertSql = "UPDATE alerts SET `LastAlerted` = %s WHERE
`EmailAddress` = %s"
677         UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
678         Cursor.execute(UpdateAlertSql, UpdateAlertValues)
679         db.commit()
680
681     else:
682         window3["-NOTIFICATION-"].update(visible=False)
683
684 window1_run()

```